A.R.T

Automatic Multi-Step
Reasoning and Tool-Use .

By

Tanishk Wankhade (Research Mentee)
Jamie Robbins (MIT 26)

Reflexion: Language Agents with Verbal Reinforcement Learning

A technical analysis of Shinn, Cassano, Berman, Gopinath, Narasimhan and Yao (arXiv:2303.11366v4, October 2023)

1. Problem statement

Large language models are increasingly deployed as goal-directed agents that act in external environments: text games, compilers, search APIs, web interfaces. Systems such as ReAct, SayCan, Toolformer, HuggingGPT and WebGPT demonstrate that an LLM core can generate both reasoning traces and executable actions. What these systems lack is an efficient mechanism for learning from trial and error. The canonical approach, reinforcement learning with gradient descent on model parameters, is poorly matched to this setting for three reasons.

- **Sample and compute cost.** Policy-gradient and value-based methods typically require very large numbers of episodes and full or partial fine-tuning of a model with billions of parameters.
- **Parameter access.** Frontier models are frequently available only through inference APIs, making weight updates impossible for most practitioners.
- **Credit assignment.** Environment feedback is usually sparse and scalar (task success or failure at the end of a long trajectory). A single reward provides little information about which of many intermediate actions caused the failure, which is the classical credit assignment problem described by Sutton and Barto.

Consequently, prior language agents have been limited to in-context learning from fixed few-shot exemplars. When such an agent fails and is re-run, it has no persistent representation of the failure and tends to reproduce the same error. The paper asks whether an agent can improve substantially over a small number of trials, within a single task, without any parameter updates.

2. Core contribution

Reflexion replaces the scalar reward update with a **linguistic** one. After each trial, the evaluation signal (binary, scalar, or free-form) is passed together with the trajectory to a self-reflection model, which produces a natural-language diagnosis of the failure and a proposed corrective plan. This text is appended to an episodic memory buffer and included in the agent's context on subsequent trials.

The authors describe this reflective text as a *semantic gradient*: like a numerical gradient, it supplies a direction of improvement, but expressed in the same representational space the policy operates in (language). The policy is therefore parameterised not only by the frozen LLM weights but also by the contents of memory:

```
pi_theta(a_i | s_i),   theta = { M_a, mem }
```

Learning, in this formulation, is the process of changing *mem* rather than changing the weights of M_a. The stated contributions are: (i) the Reflexion paradigm itself; (ii) empirical evidence that self-reflection is useful for learning complex tasks in a handful of trials; (iii) LeetcodeHardGym, a new code generation environment of 40 hard problems in 19 languages; and (iv) state-of-the-art pass@1 on several code generation benchmarks at the time of publication.

3. Positioning relative to prior work

Self-Refine (Madaan et al.) iteratively critiques and rewrites a single generation, but is confined to single-step tasks and has no persistent memory across attempts. **Self-evaluation guided beam search** (Xie et al.) uses the model's own scoring to guide decoding in decision-making tasks, again without memory. Work by Pryzant et al. performs prompt optimisation via textual "gradients" but likewise targets single generations.

In code generation, **AlphaCode** filters samples against hidden test cases; **CodeT** scores candidates using self-generated tests but has no learning step; **Self-Debugging** and **CodeRL** repair programs using execution feedback, but both rely on ground-truth tests, which disqualifies them from honest pass@1 reporting. Reflexion's distinguishing combination is: self-generated tests, multi-language support, and an explicit self-reflection step whose output persists across trials.

4. Architecture

Reflexion is modular and uses three components, which may share the same underlying LLM with different prompts.

Component	Symbol	Function
Actor	M_a	Generates text and actions conditioned on the current state and on memory. Instantiated as Chain-of-Thought or ReAct depending on the task.
Evaluator	M_e	Maps a trajectory to a reward $r_t = M_e(\text{tau}_t)$. Implemented as exact match, hand-written heuristics, an LLM classifier, or self-generated unit tests.
Self-Reflection	M_{sr}	Takes (tau_t, r_t, mem) and produces reflection sr_t: a specific explanation of what failed and what to do differently.

4.1 Memory

The design distinguishes two memory scales. **Short-term memory** is the current trajectory, i.e. the full sequence of actions and observations within one trial. **Long-term memory** is the list of reflections accumulated across trials. Because context windows are finite, long-term memory is bounded to a maximum of Omega stored experiences, set to 1 to 3 in practice and implemented as a sliding window. The combination is intended to give the actor both

fine-grained recent context and distilled lessons from earlier attempts.

4.2 Algorithm

```
Initialize M_a, M_e, M_sr ;  theta = {M_a, mem}
tau_0  <- rollout(pi_theta)
r_0    <- M_e(tau_0)
sr_0   <- M_sr(tau_0, r_0)
mem    <- [sr_0] ;  t <- 0
while M_e does not pass and t < max_trials:
    tau_t <- rollout(pi_theta)          # actor conditions on mem
    r_t   <- M_e(tau_t)
    sr_t  <- M_sr(tau_t, r_t, mem)
    mem.append(sr_t)  ;  truncate to last Omega entries
    t <- t + 1
```

4.3 How the reflection addresses credit assignment

Consider a multi-step task where action a_i is a mistake that leads to subsequent errors a_{i+1} and a_{i+2}, and the only feedback is a terminal failure. A scalar update cannot localise the error. The self-reflection model, however, can read the full trajectory and state explicitly that a_i should have been replaced by a'_i, which would have led to a'_{i+1} and a'_{i+2}. On the next trial the actor is conditioned on this statement and can select a'_i at the relevant step. In effect the LLM performs credit assignment by reasoning over the trajectory, which is only possible because the model can introspect over a human-readable record of its own behaviour.

4.4 Claimed advantages and acknowledged costs

Relative to policy- or value-based RL, the authors claim Reflexion is lightweight (no fine-tuning), supports richer feedback than scalar or vector rewards, yields an interpretable episodic memory, and provides explicit hints for future episodes. The costs are equally explicit: performance depends entirely on the LLM's self-evaluation ability (or the quality of heuristics), and there is no formal convergence guarantee.

5. Experiment I: sequential decision-making (ALFWorld)

ALFWorld is a suite of text-based interactive environments built on TextWorld. The agent receives a room description and a goal such as "clean some pan and put it in countertop" and issues text commands. The evaluation covers 134 environments across six task types (finding hidden objects, relocating objects, and manipulating objects with other objects). The actor is ReAct with two domain-specific few-shot trajectories, following Yao et al.

5.1 Evaluation signal

Since the environment only reports task completion, a self-evaluation step is required to decide when to reflect. Two variants are tested: an LLM binary classifier, and a hand-written heuristic. The heuristic triggers reflection if (a) the agent repeats the same action with the same observation for more than three cycles, interpreted as hallucination, or (b) the trajectory exceeds 30 actions, interpreted as inefficient planning. In baseline runs, a triggered reflection simply resets the environment; in Reflexion runs the agent writes a reflection first. Memory is

truncated to the last three reflections.

5.2 Results

Configuration	Outcome
ReAct only	Plateaus near 0.75 of environments; no gains between trials 6 and 7
ReAct + Reflexion (heuristic)	130 / 134 solved after 12 trials; +22 points absolute
ReAct + Reflexion (GPT classifier)	Slightly below heuristic, same qualitative curve

5.3 Error analysis

The dominant baseline failure mode is a belief-state error: the agent believes it holds an object it does not hold, then executes a long chain of actions premised on that belief and cannot backtrack. The ReAct-only agent converges to a hallucination rate of roughly 22% with no recovery over trials. Under Reflexion, both hallucination and inefficient-planning failures decline toward zero. The authors identify two mechanisms: reflection localises an *early* error inside a long trajectory and proposes a revised plan, and the accumulated memory allows systematic coverage of a search space too large to exhaust in one trial (e.g. many containers). The learning curve shows a sharp improvement between the first two trials followed by steady gains, consistent with both mechanisms operating.

6. Experiment II: reasoning (HotPotQA)

HotPotQA contains 113k Wikipedia-based multi-hop questions requiring reasoning across several supporting documents. Experiments use 100 questions. Three actor configurations isolate different capabilities:

- **CoT, Q to A:** chain-of-thought reasoning with no retrieval (6-shot).
- **CoT (GT), (Q, C_{gt}) to A:** the ground-truth context is supplied, isolating reasoning over long text from retrieval.
- **ReAct:** the agent retrieves Wikipedia content via API and reasons step by step (2-shot).

The evaluator is exact match, producing a binary signal. Self-reflection uses 2-shot prompting and memory is bounded at three. In Reflexion runs the agent retries a task until three consecutive failures.

6.1 Results

The most striking observation concerns the baselines: ReAct-only, CoT-only and CoT (GT)-only fail to solve *any* task on retry that they failed on the first trial, at temperature 0.7. Stochastic resampling alone does not produce improvement. Reflexion yields approximately +20 points overall. Even with ground-truth context, CoT (GT) fails 39% of questions on the first pass; Reflexion recovers an additional 14 points without access to the answer.

Actor + model (100 questions, Appendix A)	Baseline	Reflexion	Delta

CoT (GT) + text-davinci-003	0.60	0.77	+0.17
CoT (GT) + gpt-3.5-turbo	0.57	0.71	+0.14
CoT (GT) + gpt-4	0.68	0.80	+0.12
ReAct + text-davinci-003	0.30	0.55	+0.25
ReAct + gpt-3.5-turbo	0.26	0.38	+0.12
ReAct + gpt-4	0.39	0.51	+0.12

6.2 Ablation: reflection versus episodic memory

A natural objection is that gains come merely from showing the agent its previous trajectory. The authors test this with CoT (GT) under three conditions: no memory; episodic memory (EPM), where the most recent trajectory is included; and EPM plus a first-person self-reflection. Self-reflection adds an **8-point absolute improvement over EPM alone**. The interpretation is that refinement conditioned on a raw trajectory is weaker than refinement conditioned on an explicit verbal diagnosis, supporting the claim that the reflection step, not just exposure to past attempts, is causally important.

7. Experiment III: programming

Code generation is evaluated on HumanEval and MBPP (function-body synthesis from natural-language specifications), their Rust translations produced with the MultiPL-E compiler suite, and LeetcodeHardGym, 40 hard-rated LeetCode problems released after 8 October 2022, the stated pre-training cutoff of GPT-4, to reduce contamination. Including Rust tests whether the method transfers to a compiled, statically typed language.

7.1 Internal evaluation via self-generated tests

Because hidden tests may not be used if pass@1 is to be reported honestly, the evaluator is built from the agent's own tests. Chain-of-thought prompting produces a diverse set of candidate unit tests with natural-language descriptions; candidates are filtered for syntactic validity by attempting to construct an abstract syntax tree; up to $n = 6$ are sampled to form the suite $T = \{t_0, ..., t_n\}$. The implementation is iterated against T with reflection, and only the final program is submitted. Memory is limited to a single experience.

7.2 Results

Benchmark	**Prior SOTA pass@1**	**GPT-4 pass@1**	**Reflexion pass@1**
HumanEval (Python)	65.8 (CodeT + GPT-3.5)	80.1	91.0
HumanEval (Rust)	-	60.0	68.0
MBPP (Python)	67.7 (CodeT + Codex)	80.1	77.1
MBPP (Rust)	-	70.9	75.4
LeetcodeHard (Python)	-	7.5	15.0

7.3 Test-suite reliability as the binding constraint

The one regression, MBPP Python, is explained by analysing the internal test suite as a classifier of solution correctness. A **false positive** occurs when all self-generated tests pass on an incorrect solution: the agent stops early and submits a wrong answer. A **false negative** occurs when a test fails on a correct solution. False negatives are comparatively benign, because reflection can identify the faulty test and leave the code unchanged; false positives are fatal, because no further trial is triggered.

Benchmark	Base	Reflexion	TP	FN	FP	TN
HumanEval (PY)	0.80	0.91	0.99	0.40	0.01	0.60
MBPP (PY)	0.80	0.77	0.84	0.59	0.16	0.41
HumanEval (RS)	0.60	0.68	0.87	0.37	0.13	0.63
MBPP (RS)	0.71	0.75	0.84	0.51	0.16	0.49

The false-positive rate, P(solution incorrect | tests pass), is 16.3% on MBPP Python versus 1.4% on HumanEval Python, despite similar baseline accuracy. The result illustrates the paper's central dependency: Reflexion amplifies the quality of its evaluator, and a weak evaluator yields confident errors.

7.4 Component ablation (HumanEval Rust, 50 hardest problems, GPT-4)

Configuration	Test generation	Self-reflection	pass@1
Base model	no	no	0.60
Test generation omitted	no	yes	0.52
Self-reflection omitted	yes	no	0.60
Full Reflexion	yes	yes	0.68

Without tests, the agent cannot determine whether its current implementation is already correct, so it runs all iterations and makes harmful edits, falling below baseline. With tests but no verbal reflection, the compiler and test failures expose syntax and logic errors, but the subsequent edits do not reflect them, producing no gain. The authors argue this undermines "blind" trial-and-error debugging approaches on harder tasks: error *detection* and error *interpretation* are each necessary and neither is sufficient.

8. Limitations

8.1 Dependence on model scale

Appendix A reports HumanEval Python with the open model starchat-beta: 0.26 pass@1 for both baseline and Reflexion (averaged over eight trials). Producing useful self-corrections appears to be an emergent capability of larger models; below some capability threshold the reflections carry no usable signal.

8.2 Exploration-heavy tasks

On WebShop, an e-commerce navigation benchmark, a two-shot ReAct + Reflexion agent across 100 environments showed no meaningful improvement after four trials (roughly 0.33 to 0.35, against a ReAct baseline of about 0.34) and generated unhelpful reflections. The authors attribute this to the need for diverse, creative exploration: in ALFWorld the admissible actions are visible in observations, and Wikipedia search tolerates imprecise queries, whereas e-commerce search is sensitive to ambiguous natural-language queries. Reflexion corrects identifiable mistakes well but does not itself generate novel exploration strategies, and can settle in local minima.

8.3 Other constraints

- **Memory structure.** Long-term memory is a small sliding window. The authors propose vector databases or SQL stores as future extensions.
- **Test-driven evaluation limits.** Accurate input-output specifications are hard to write for non-deterministic, impure, hardware-dependent, or concurrent functions.
- **Evaluator reliability.** All gains are mediated by the evaluator; noisy heuristics or flaky tests propagate directly into the learning signal.
- **Execution safety.** Generated code is executed without validation, so isolated sandboxes are recommended.

9. Interpretation and significance

Reflexion reframes a component of learning as an in-context, text-mediated process. The policy improves because its conditioning context improves, which makes adaptation cheap, fast (on the order of a dozen trials) and available for closed models. Because the learned state is natural language, it is directly inspectable: a human can audit what the agent believes it did wrong, and the authors suggest such reflections could be monitored to verify intent before an agent invokes a risky tool, an interpretability property unavailable in conventional black-box policy optimisation.

The experimental record supports a fairly precise conclusion. Gains require three conditions: a model strong enough to diagnose its own failures, an evaluator reliable enough to signal failure correctly, and a task where failures are attributable to identifiable decisions rather than to insufficient exploration. Where these hold, verbal reflection outperforms both blind resampling and exposure to raw past trajectories. The attempt, evaluate, reflect, retry loop has since become a standard pattern in coding agents and agentic frameworks, and the paper's proposed future directions include importing value learning and off-policy exploration from classical RL into the natural-language setting.

Adapted from

Reflexion

By Noah Shinn